

UNIVERSIDAD PRIVADA DOMINGO SAVIO

FACULTAD DE INGENIERIA

INGENIERIA EN SISTEMAS



UNIVERSIDAD PRIVADA
DOMINGO SAVIO

GESTIÓN DE VERSIONES CON GIT Y GITHUB

LABORATORIO GUIADO DE INGENIERÍA DE SOFTWARE II

Integrantes:

Ronald Machaca Cori

Docente:

Ph. D. Luigi Antequera Tamari

La Paz – Bolivia

Julio - 2026

Laboratorio Guiado

Ingeniería de Software II

Primeros pasos con Git y GitHub usando JavaScript

Duración: 60-90 minutos

Objetivo: Aprender los conceptos básicos de Git y GitHub mediante un proyecto sencillo en JavaScript.

Al finalizar este laboratorio el estudiante podrá:

- Instalar y configurar Git.
- Crear un repositorio local.
- Crear un repositorio en GitHub.
- Registrar cambios mediante commits.
- Enviar un proyecto a GitHub.
- Recuperar el historial del proyecto.
- Ejecutar un programa JavaScript con Node.js.

Caso de Estudio

Una empresa desea desarrollar un pequeño sistema que calcule el promedio de tres notas de un estudiante.

Durante el desarrollo se realizarán varias mejoras y cada una será registrada mediante Git.

Paso 1. Verificar instalaciones

Verificar Git

Abrir una terminal y ejecutar:

```
git --version
```

Resultado esperado

```
git version 2.xx.x
```

Verificar Node.js

```
node -v
```

Resultado esperado

```
v20.x.x
```

Verificar npm

```
npm -v
```

```
Administrator: Windows PowerShell

PS C:\Users\RON\Desktop\GITHUB> git --version
git version 2.52.0.windows.1
PS C:\Users\RON\Desktop\GITHUB> node -v
v24.12.0
PS C:\Users\RON\Desktop\GITHUB> npm -v
11.6.2
```

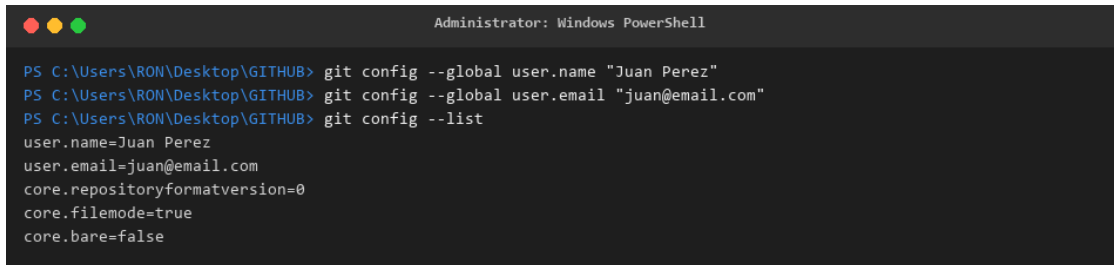
Paso 2. Configurar Git

Ejecutar una sola vez.

```
git config --global user.name "Juan Perez"  
git config --global user.email "juan@email.com"
```

Verificar

```
git config --list
```



```
Administrator: Windows PowerShell  
  
PS C:\Users\RON\Desktop\GITHUB> git config --global user.name "Juan Perez"  
PS C:\Users\RON\Desktop\GITHUB> git config --global user.email "juan@email.com"  
PS C:\Users\RON\Desktop\GITHUB> git config --list  
user.name=Juan Perez  
user.email=juan@email.com  
core.repositoryformatversion=0  
core.filemode=true  
core.bare=false
```

Paso 3. Crear el proyecto

Crear una carpeta

```
mkdir PromedioNotas
```

Entrar

```
cd PromedioNotas
```

Inicializar Node.js

```
npm init -y
```

Se creará automáticamente

```
package.json
```

```
Administrator: Windows PowerShell

PS C:\Users\RON\Desktop\GITHUB> mkdir PromedioNotas
PS C:\Users\RON\Desktop\GITHUB> cd PromedioNotas
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> npm init -y
Wrote to C:\Users\RON\Desktop\GITHUB\PromedioNotas\package.json:
{
  "name": "promedionotas",
  "version": "1.0.0",
  "main": "index.js",
  "license": "ISC"
}
```

Paso 4. Crear el proyecto JavaScript

Crear el archivo

app.js

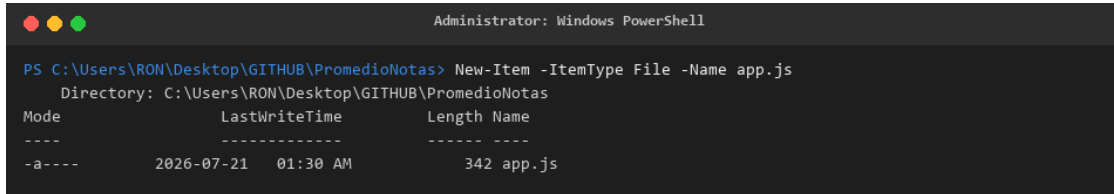
Agregar el siguiente código.

```
// =====
// Sistema de Promedio de Notas
// =====

const nota1 = 70;
const nota2 = 85;
const nota3 = 90;

const promedio = (nota1 + nota2 + nota3) / 3;
```

```
console.log("Sistema de Promedios");  
console.log("                ");  
console.log("Nota 1:", nota1);  
console.log("Nota 2:", nota2);  
console.log("Nota 3:", nota3);  
console.log("Promedio:", promedio.toFixed(2));
```



```
Administrator: Windows PowerShell  
  
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> New-Item -ItemType File -Name app.js  
Directory: C:\Users\RON\Desktop\GITHUB\PromedioNotas  
Mode                LastWriteTime         Length Name  
----                -  
-a-----          2026-07-21   01:30 AM             342 app.js
```

Paso 5. Ejecutar el programa

En la terminal ejecutar

```
node app.js
```

Resultado esperado

Sistema de Promedios

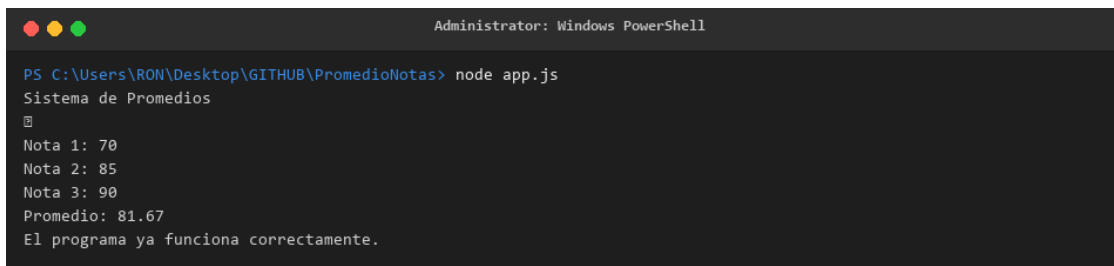
Nota 1: 70

Nota 2: 85

Nota 3: 90

Promedio: 81.67

El programa ya funciona correctamente.



```
Administrator: Windows PowerShell  
  
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> node app.js  
Sistema de Promedios  
[  
Nota 1: 70  
Nota 2: 85  
Nota 3: 90  
Promedio: 81.67  
El programa ya funciona correctamente.
```

Paso 6. Inicializar Git

Dentro de la carpeta ejecutar

```
git init
```

Resultado

```
Initialized empty Git repository
```

Verificar el estado

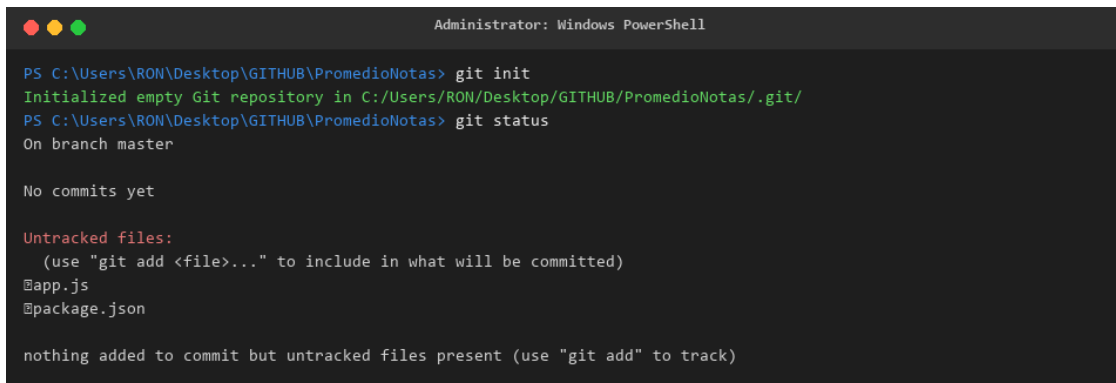
```
git status
```

Resultado

```
Untracked files:
```

```
app.js  
package.json
```

Git informa que existen archivos que todavía no están siendo administrados.



```
Administrator: Windows PowerShell  
  
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git init  
Initialized empty Git repository in C:/Users/RON/Desktop/GITHUB/PromedioNotas/.git/  
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git status  
On branch master  
  
No commits yet  
  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
   app.js  
  package.json  
  
nothing added to commit but untracked files present (use "git add" to track)
```

Paso 7. Agregar archivos

Agregar todos los archivos

```
git add .
```

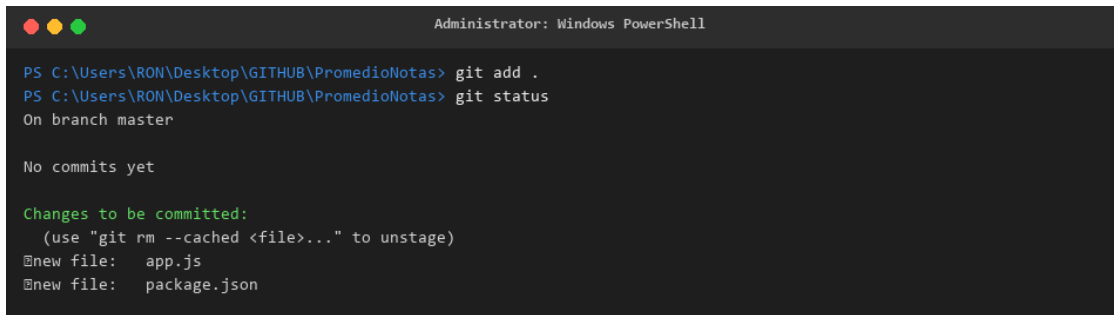
Verificar nuevamente

```
git status
```

Resultado esperado

Changes to be committed

Los archivos están listos para guardarse en el historial.



```
Administrator: Windows PowerShell

PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git add .
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
   new file:   app.js
   new file:   package.json
```

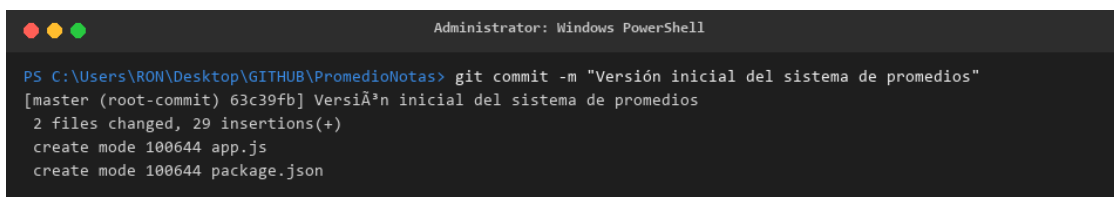
Paso 8. Crear el primer Commit

Guardar el proyecto

```
git commit -m "Versión inicial del sistema de promedios"
```

Resultado esperado

1 file changed...



```
Administrator: Windows PowerShell

PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git commit -m "Versión inicial del sistema de promedios"
[master (root-commit) 63c39fb] Versión inicial del sistema de promedios
 2 files changed, 29 insertions(+)
 create mode 100644 app.js
 create mode 100644 package.json
```


Paso 9. Ver el historial

```
git log
```

Resultado

```
commit a8f45....
```

```
Author: Juan Perez
```

```
Date: ...
```

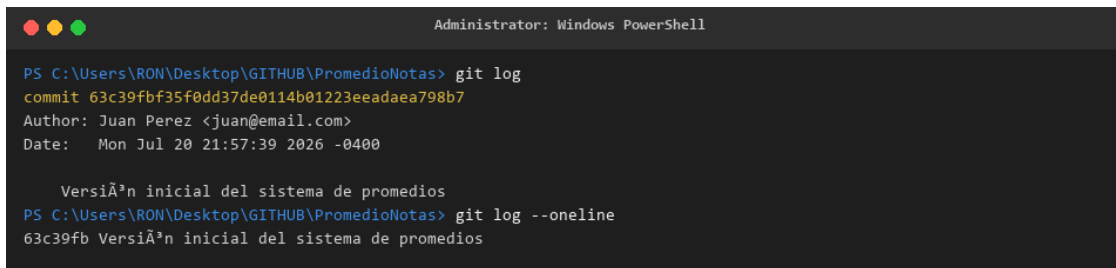
```
Versión inicial del sistema de promedios
```

También puede utilizar

```
git log --oneline
```

Resultado

```
a8f45 Versión inicial del sistema de promedios
```



```
Administrator: Windows PowerShell

PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git log
commit 63c39fbf35f0dd37de0114b01223eeadaea798b7
Author: Juan Perez <juan@email.com>
Date: Mon Jul 20 21:57:39 2026 -0400

    VersiÃ³n inicial del sistema de promedios
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git log --oneline
63c39fb VersiÃ³n inicial del sistema de promedios
```

Paso 10. Realizar un cambio

Modificar el archivo

```
const nota1 = 95;
```

Guardar.

Ejecutar nuevamente

```
node app.js
```

Resultado

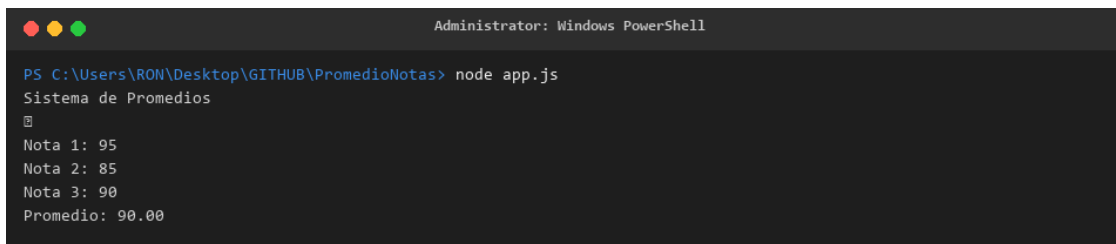
```
Sistema de Promedios
```

```
Nota 1: 95
```

```
Nota 2: 85
```

```
Nota 3: 90
```

```
Promedio: 90.00
```



```
Administrator: Windows PowerShell

PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> node app.js
Sistema de Promedios
[
  Nota 1: 95
  Nota 2: 85
  Nota 3: 90
  Promedio: 90.00
]
```

Paso 11. Verificar cambios

```
git status
```

Git detectará que el archivo fue modificado.

Ver exactamente qué cambió

```
git diff
```

Resultado aproximado

70

95

```
Administrator: Windows PowerShell

PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
   modified:   app.js

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git diff
diff --git a/app.js b/app.js
index 1a36001..9b2bc9a 100644
--- a/app.js
+++ b/app.js
@@ -2,7 +2,7 @@
 // Sistema de Promedio de Notas
 // =====

-const nota1 = 70;
+const nota1 = 95;
 const nota2 = 85;
 const nota3 = 90;
```

Paso 12. Registrar el cambio

Agregar

```
git add .
```

Guardar

```
git commit -m "Actualiza la nota del primer estudiante"
```

Ver historial

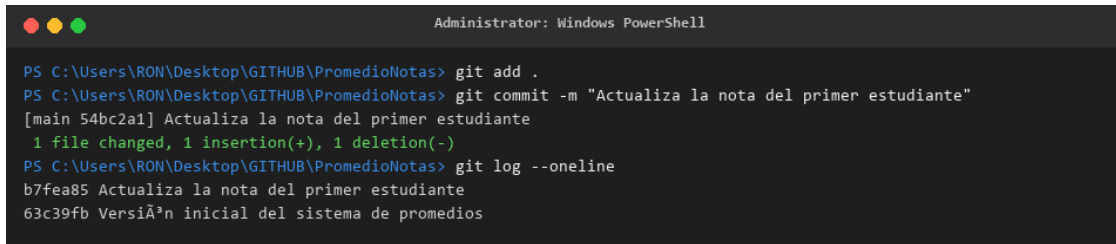
```
git log --oneline
```

Resultado

54bc2 Actualiza la nota del primer estudiante

9af33 Versión inicial del sistema de promedios

Ahora el proyecto tiene dos versiones.



```
Administrator: Windows PowerShell

PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git add .
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git commit -m "Actualiza la nota del primer estudiante"
[main 54bc2a1] Actualiza la nota del primer estudiante
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git log --oneline
b7fea85 Actualiza la nota del primer estudiante
63c39fb VersiÃ³n inicial del sistema de promedios
```

Paso 13. Crear el repositorio en GitHub

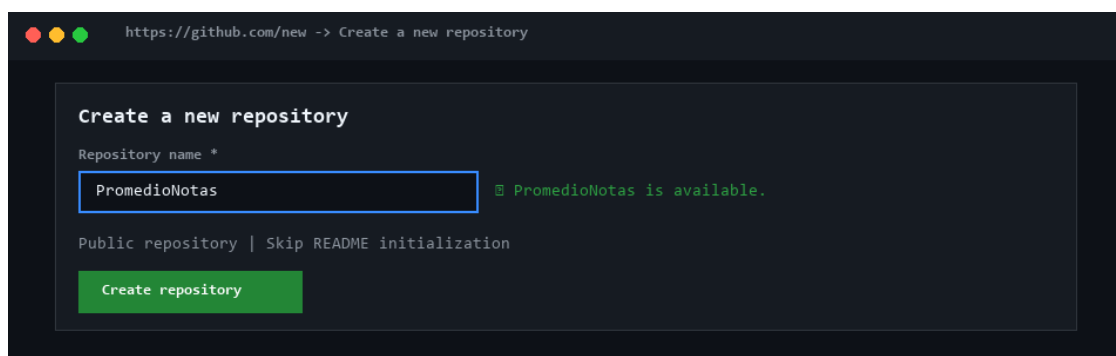
Ingresar a:

<https://github.com>

Crear un nuevo repositorio llamado

PromedioNotas

No agregar README.



Paso 14. Conectar Git con GitHub

Copiar la URL del repositorio.

Ejemplo

```
https://github.com/usuario/PromedioNotas.git
```

Ejecutar

```
git remote add origin  
https://github.com/usuario/PromedioNotas.git
```

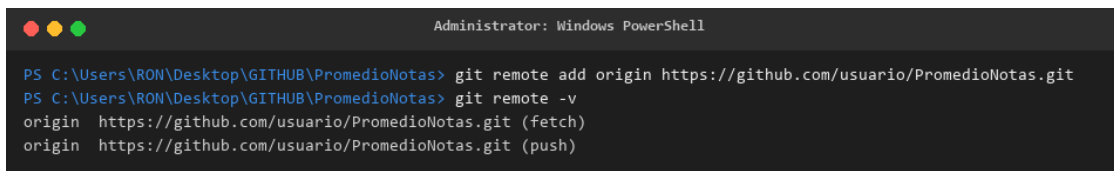
Verificar

```
git remote -v
```

Resultado

```
origin
```

```
origin
```



```
Administrator: Windows PowerShell  
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git remote add origin https://github.com/usuario/PromedioNotas.git  
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git remote -v  
origin https://github.com/usuario/PromedioNotas.git (fetch)  
origin https://github.com/usuario/PromedioNotas.git (push)
```

Paso 15. Enviar el proyecto

Cambiar la rama principal

```
git branch -M main
```

Enviar

```
git push -u origin main
```

Ingresar a GitHub.

Ahora el proyecto aparecerá publicado.

```
Administrator: Windows PowerShell

PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git branch -M main
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 580 bytes | 580.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0
To https://github.com/usuario/PromedioNotas.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Paso 16. Crear una nueva versión

Modificar el programa.

Agregar

```
console.log("                                ");
console.log("Fin del programa");
```

Ejecutar

```
node app.js
```

Resultado

Sistema de Promedios

Nota 1: 95

Nota 2: 85

Nota 3: 90

Promedio: 90.00

Fin del programa

Guardar

```
git add .
```

Crear commit

```
git commit -m "Se agrega mensaje final"
```

Enviar

git push

```
Administrator: Windows PowerShell

PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> node app.js
Sistema de Promedios
[
  Nota 1: 95
  Nota 2: 85
  Nota 3: 90
  Promedio: 90.00
]
Fin del programa
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git add .
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git commit -m "Se agrega mensaje final"
[main a912e34] Se agrega mensaje final
 1 file changed, 2 insertions(+)
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git push
Everything up-to-date / Pushed main -> origin/main
```

Paso 17. Crear una etiqueta (Tag)

Crear la versión

git tag v1.0.0

Ver etiquetas

git tag

Resultado

v1.0.0

Enviar

git push origin v1.0.0

```
Administrator: Windows PowerShell

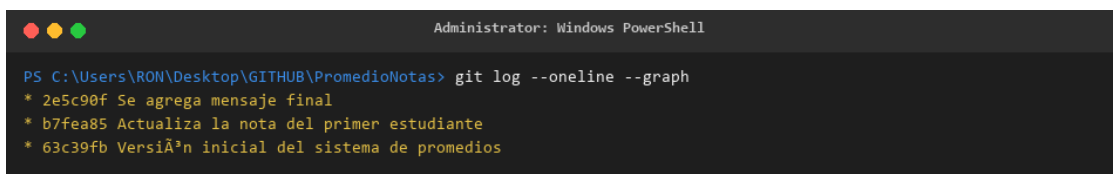
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git tag v1.0.0
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git tag
v1.0.0
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git push origin v1.0.0
To https://github.com/usuario/PromedioNotas.git
 * [new tag]          v1.0.0 -> v1.0.0
```


Paso 18. Ver el historial gráfico

```
git log --oneline --graph
```

Resultado

- * Se agrega mensaje final
- * Actualiza la nota del primer estudiante
- * Versión inicial del sistema



```
Administrator: Windows PowerShell

PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git log --oneline --graph
* 2e5c90f Se agrega mensaje final
* b7fea85 Actualiza la nota del primer estudiante
* 63c39fb VersiÃn inicial del sistema de promedios
```

Estructura final del proyecto

PromedioNotas/

|

|— app.js

|— package.json

Actividad para el estudiante

Modificar el programa para que ahora calcule el promedio de **cinco** notas.

Registrar el cambio utilizando Git.

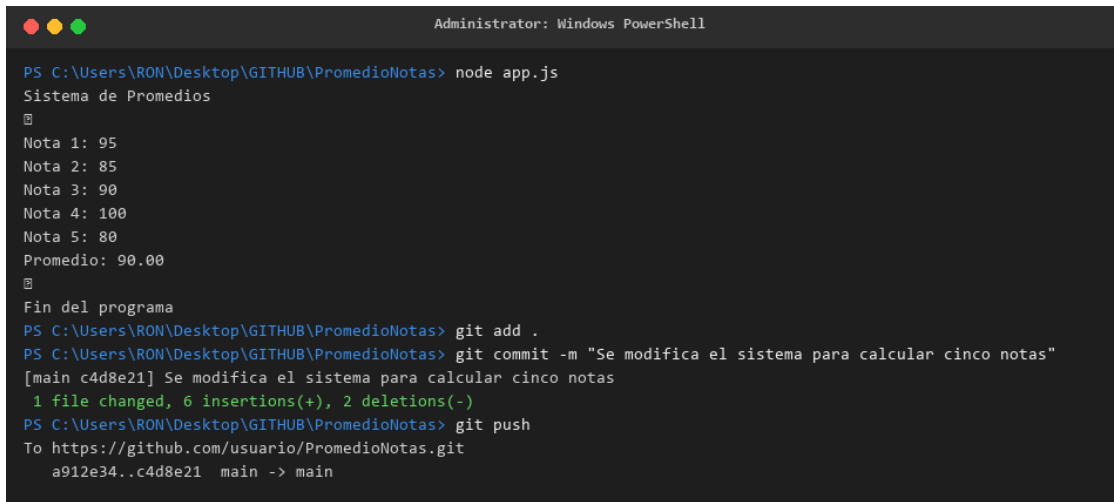
El mensaje del commit debe ser descriptivo.

Ejemplo

```
git commit -m "Se modifica el sistema para calcular cinco notas"
```

Enviar los cambios a GitHub.

```
git push
```



```
Administrator: Windows PowerShell

PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> node app.js
Sistema de Promedios
[
  Nota 1: 95
  Nota 2: 85
  Nota 3: 90
  Nota 4: 100
  Nota 5: 80
  Promedio: 90.00
]
Fin del programa
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git add .
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git commit -m "Se modifica el sistema para calcular cinco notas"
[main c4d8e21] Se modifica el sistema para calcular cinco notas
1 file changed, 6 insertions(+), 2 deletions(-)
PS C:\Users\RON\Desktop\GITHUB\PromedioNotas> git push
To https://github.com/usuario/PromedioNotas.git
a912e34..c4d8e21 main -> main
```

Preguntas de Reflexión

1. ¿Qué diferencia existe entre Git y GitHub?

Git es un sistema de control de versiones distribuido que se ejecuta localmente en la computadora para registrar y gestionar el historial de cambios del código, mientras que GitHub es una plataforma en la nube que hospeda repositorios creados con Git para facilitar el respaldo, la colaboración remota y la gestión de proyectos.

2. ¿Qué función cumple un commit?

Un commit actúa como un punto de restauración o captura instantánea (snapshot) del estado del proyecto en un instante dado, guardando de forma permanente en el historial de Git todos los cambios del staging area junto a su autor, fecha y mensaje descriptivo.

3. ¿Por qué es importante escribir mensajes descriptivos en los commits?

Los mensajes descriptivos permiten a los desarrolladores comprender rápidamente el motivo y alcance de cada modificación sin inspeccionar el código fuente línea por línea, agilizando las auditorías y la depuración del proyecto a lo largo del tiempo.

4. ¿Qué información muestra el comando git status?

El comando git status despliega el estado del directorio de trabajo y del área de preparación (staging area), detallando la rama actual, los archivos modificados, los archivos sin seguimiento (untracked) y los cambios listos para ser confirmados en un commit.

5. ¿Qué utilidad tiene git diff?

El comando git diff muestra la comparación detallada línea por línea entre los archivos del directorio de trabajo local y las versiones guardadas en el historial o staging area, permitiendo revisar minuciosamente las modificaciones antes de guardarlas.

6. ¿Qué ocurre si modificamos un archivo y no ejecutamos git add?

Si se modifica un archivo y se omite el comando git add, las alteraciones permanecerán únicamente en el directorio de trabajo local sin pasar al área de preparación, por lo que no quedarán registradas en el siguiente commit.

7. ¿Para qué sirve una etiqueta (tag)?

Una etiqueta (tag) en Git es una marca fija e inalterable asignada a un commit específico del historial para identificar e independizar versiones o lanzamientos importantes del proyecto, tales como v1.0.0 o v2.0.0.

8. ¿Qué ventajas ofrece almacenar un proyecto en GitHub?

Almacenar un proyecto en GitHub ofrece un respaldo seguro en la nube, habilita el trabajo colaborativo distribuido mediante ramas y Pull Requests, y proporciona integración continua, control de acceso y seguimiento de errores en un solo lugar.

9. ¿Qué problemas podrían surgir si varias personas modifican un proyecto sin utilizar Git?

Modificar un proyecto en equipo sin Git genera sobrescrituras accidentales de código, pérdida irrecoverable de cambios, falta de atribución sobre la autoría de errores y enormes dificultades técnicas para unificar el trabajo de múltiples desarrolladores.

10. ¿Qué beneficios aporta el control de versiones en proyectos de software?

El control de versiones aporta trazabilidad absoluta sobre la evolución del código, la flexibilidad de revertir fallos hacia versiones estables anteriores, la capacidad de desarrollar funciones en paralelo sin alterar el código principal y mayor seguridad en las entregas.

Comandos utilizados en este laboratorio

Acción	Comando
Ver versión de Git	git --version
Ver versión de Node.js	node -v
Crear proyecto Node.js	npm init -y
Ejecutar programa	node app.js
Inicializar Git	git init
Ver estado	git status

Agregar archivos	<code>git add .</code>
Guardar cambios	<code>git commit -m "mensaje"</code>
Ver historial	<code>git log</code>

Acción	Comando
Ver historial resumido	<code>git log --oneline</code>
Ver diferencias	<code>git diff</code>
Agregar repositorio remoto	<code>git remote add origin URL</code>
Ver repositorios remotos	<code>git remote -v</code>
Cambiar rama principal	<code>git branch -M main</code>
Enviar proyecto a GitHub	<code>git push -u origin main</code>
Crear etiqueta	<code>git tag v1.0.0</code>
Enviar etiquetas	<code>git push origin v1.0.0</code>

Resultado esperado

Al finalizar este laboratorio, el estudiante habrá creado un proyecto JavaScript funcional, lo habrá versionado con Git, publicado en GitHub, registrado múltiples versiones mediante *commits* y etiquetado una versión estable.